

Lista de Monitoria 1

Análise de Algoritmos

SCC-0201 — Introdução à Ciência da Computação II

Parte 1 — Complexidade Assintótica

Questão 1. Explique com suas palavras a diferença entre o método empírico e o método analítico de avaliação de algoritmos. Por que a comunidade de computação prefere comparar *algoritmos* e não *programas*?

Questão 2. Considere as funções $f(n) = n^2 + n$ e $g(n) = 1000n$.

(a) Calcule $f(n)$ e $g(n)$ para $n = 1, 10, 100, 1000, 10000$.

(b) A partir de qual valor de n temos $f(n) \geq g(n)$?

(c) Mostre formalmente que $1000n = O(n^2)$, exibindo as constantes c e n_0 .

Questão 3. Para cada par de funções abaixo, diga se $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$, $f(n) = \Theta(g(n))$ ou nenhuma das opções. Justifique.

(a) $f(n) = 5n^3 + 2n$, $g(n) = n^3$

(b) $f(n) = n \log n$, $g(n) = n^2$

(c) $f(n) = 2^n$, $g(n) = n^{10}$

(d) $f(n) = \log n$, $g(n) = \log^2 n$

Questão 4. Mostre, usando a definição formal de Θ , que

$$g(n) = \frac{n^2}{3} - 2n$$

é $\Theta(n^2)$. Encontre valores válidos para c_1 , c_2 e n_0 .

Questão 5. Coloque as funções abaixo em ordem crescente de taxa de crescimento assintótica:

$$n^2, \quad \log n, \quad n!, \quad 2^n, \quad n \log n, \quad n^3, \quad n, \quad \log^2 n, \quad \sqrt{n}$$

Questão 6. Um algoritmo tradicional roda em $O(n^{1.5})$ e um algoritmo novo roda em $O(n \log n)$. Mostre matematicamente qual dos dois cresce mais devagar e explique por que essa escolha pode ser decisiva para uma empresa que processa grandes volumes de dados.

Questão 7. Sejam $T_1(n) = O(f(n))$ e $T_2(n) = O(g(n))$. Explique (com um exemplo de código) por que:

(a) $T_1(n) + T_2(n) = O(\max(f(n), g(n)))$

(b) $T_1(n) \cdot T_2(n) = O(f(n) \cdot g(n))$

Questão 8. Dado o trecho de código abaixo, determine sua complexidade assintótica no pior caso, contando o número de operações realizadas em função de n :

```
soma = 0
for i = 1 to n:
    for j = 1 to n:
        soma = soma + i * j
```

Questão 9. Explique a diferença entre $T_{\text{melhor}}(n)$, $T_{\text{médio}}(n)$ e $T_{\text{pior}}(n)$. Por que, na prática, costuma-se usar $T_{\text{pior}}(n)$ como referência principal na análise de algoritmos?

Parte 2 — Recorrências

Questão 10. Explique a diferença entre uma recursão que é apenas um "disfarce" de um laço simples e uma recursão do tipo dividir-e-conquistar. Dê um exemplo de cada.

Questão 11. Escreva a equação de recorrência para os seguintes algoritmos e indique sua complexidade final:

- (a) Sequência de Fibonacci (versão recursiva ingênua)
- (b) Busca binária
- (c) Mergesort

Questão 12. Utilize o **Método da Substituição** para provar, por indução matemática, que a recorrência

$$T(n) = 2T(n/2) + n, \quad T(1) = 1$$

é $O(n \log n)$.

Questão 13. Utilize o **Método da Árvore de Recursão** para resolver a recorrência do Mergesort:

$$T(n) = 2T(n/2) + n$$

Desenhe (ou descreva) os níveis da árvore, indicando o custo de cada nível e a altura total da árvore, chegando à complexidade final.

Questão 14. Resolva as recorrências abaixo utilizando o **Método Mestre**. Para cada uma, identifique os valores de a , b e $f(n)$, indique em qual dos três casos ela se encaixa e apresente o resultado final de $T(n)$.

- (a) $T(n) = 4T(n/2) + n$
- (b) $T(n) = 2T(n/2) + n^2$
- (c) $T(n) = 8T(n/2) + n^3$
- (d) $T(n) = 3T(n/4) + n \log n$

Questão 15. Por que o Método Mestre não pode ser aplicado à recorrência

$$T(n) = 2T(n/2) + n/\log n ?$$

Qual é a "lacuna" entre os três casos que impede sua aplicação direta aqui?

Questão 16. Considere a recorrência da sequência de Fibonacci:

$$T(n) = T(n-1) + T(n-2) + 6$$

Explique por que essa recorrência resulta em complexidade $O(2^n)$, mesmo o problema tendo, em teoria, uma solução iterativa em $O(n)$. O que esse contraste revela sobre a importância da escolha do algoritmo, e não apenas da linguagem ou do hardware?